



CITS3402/CITS5507: Assignment 1

Semester 2, 2026

Assessed, worth **12.5%**. **Due 11:59 pm Friday 11th September.**

1 Outline

Your assignment is to develop a program to brute-force collisions for an intentionally weak hash function via a birthday attack, and to produce a report on its performance. **The project can be done individually or in a group of two.**

While this assignment is centred on programming, your submission is heavily weighted towards the report. Do not expect to receive full marks for an implementation without thorough analysis.

2 Description

The integrity or authenticity of files is often verified via *cryptographic hash functions* to confirm that two copies are identical. You may have run into this in the real world when downloading files, e.g., the SHA-256 checksum for Ubuntu Desktop 26.04 LTS:

```
echo "487f87faaf547ea30e0aba4d5b53346292571256b25333a978db1692bcee9dd2"
↪ *ubuntu-26.04-desktop-amd64.iso | shasum -a 256 --check
```

A *cryptographic hash function* maps data of arbitrary size to a fixed-length value called a *hash*. A secure cryptographic hash function has several important properties:

- **Determinism**: The same input always produces the same hash.
- **Preimage resistance**: Given a hash h , it should be computationally infeasible to find an input x such that $H(x) = h$.
- **Second-preimage resistance**: Given an input x , it should be computationally infeasible to find a different input y such that $H(x) = H(y)$.
- **Collision resistance**: It should be computationally infeasible to find any two different inputs x and y such that $H(x) = H(y)$.
- **Uniform, random-looking output**: Hash values should be well distributed and appear random, even though the function is deterministic.
- **Avalanche effect**: A small change to the input should substantially change the resulting hash.

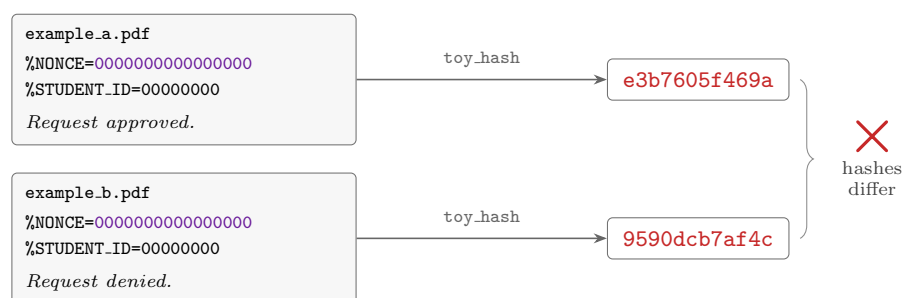
As a consequence of the above properties, when two files share the same secure cryptographic hash, you can be highly confident that they are identical. However, because hash values have a fixed length, collisions must exist, while collision resistance means that they should be difficult to find. You can learn more about [hash functions on Wikipedia](#) if you're interested, but understanding the above explanation is sufficient for this project.

Your task in project 1 is to develop a parallel program using OpenMP to brute-force collisions for an intentionally weak 48-bit hash function based on [FNV-1a](#) and [MurmurHash3](#), which we call `toy_hash`. We have specifically developed `toy_hash` for this project, and made it intentionally weak (i.e. easily exploitable) so that collisions can be brute-forced via a [birthday attack](#) on Kaya.

You have been provided with 6 pairs of specially created files (`*-{a,b}.pdf`). These PDFs contain a special header with a *nonce* field. For this project, a nonce is an arbitrary 64-bit unsigned integer stored in each file's header. Modifying it changes the file's bytes and therefore the input to the hash function. Recall that even a small change to the input should produce a substantially different hash. Each header also contains a *student number* field, which you must fill in with your own student number before you begin searching.

For each pair of files, you must find a pair of nonces that, when inserted into the file headers, cause the files to produce the same hash. Figure 1 shows what this looks like for one pair. For the simplest pairs, you should be able to brute-force a collision on your personal computer. The harder pairs may require all cores of a single Kaya node, but your job must complete within 15 minutes.

(a) As distributed



(b) After a successful search

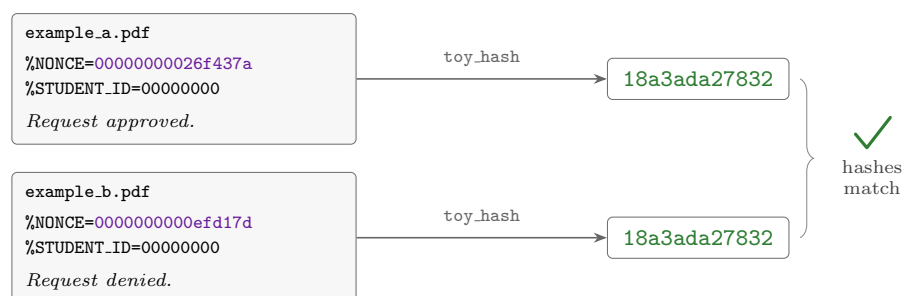


Figure 1: The goal, shown for one pair of files. Only the 16 nonce characters (purple) and the 8 student number characters may be changed. As distributed, the two documents hash to different values (a); a successful search finds a pair of nonces that makes two documents saying opposite things produce one identical 48-bit hash (b).

Note that the ability to change both nonces is important. You may wish to read about the [birthday paradox](#) and [birthday attacks](#) to understand why.

2.1 `toy_hash` reference implementation

We have provided you with an implementation of `toy_hash` in C via `toy_hash.{c,h}` and in Python via `check_toy_hash.py`. The function always returns a 48-bit hash value. You may find it useful to inspect these files, but it is not necessary to understand the implementation details or underlying hash functions for this project.

For this project, treat each call to `toy_hash` as sequential. Your implementation should focus on parallelising independent nonce trials and the collision detection logic rather than modifying the hash function itself.

2.2 File header specifications

Each provided PDF begins with this ASCII header:

```
%PDF-1.4
%NONCE=0000000000000000
%STUDENT_ID=00000000
```

Its byte layout is:

Byte offset(s)	Contents
0–8	%PDF-1.4\n
9–15	%NONCE=
16–31	16-character hexadecimal nonce
32	\n
33–44	%STUDENT_ID=
45–52	8-character student number
53	\n

Table 1: Layout of the PDF header.

Here, \n is one newline byte (0x0a). The nonce is a zero-padded, lowercase hexadecimal representation of a 64-bit unsigned integer. For example, the decimal value 42 is:

```
0000000000000002a
```

Bytes 45–52 hold your student number. Student numbers are 8 digits long. You should treat this field as a fixed-length string of exactly 8 ASCII characters rather than as a number, and copy those characters into the header verbatim. Every PDF is distributed with a placeholder value (00000000), and you must replace it with your own. If you are working in a group of two, use the student number of the member who submits to LMS.

Only bytes 16–31 and 45–52 may be changed. Both fields must be overwritten in place: do not insert or remove bytes or modify any other part of the file. Both header lines begin with %, making them invisible PDF comments, so the visible content of the file will appear unchanged.

The following functions safely update the nonce and the student number in a PDF that has been loaded into memory:

```
#include <stdint.h>
#include <stddef.h>
#include <string.h>

#define NONCE_OFFSET 16
#define NONCE_LENGTH 16
#define STUDENT_ID_OFFSET 45
#define STUDENT_ID_LENGTH 8

static void set_nonce(unsigned char *pdf, uint64_t nonce)
{
    static const char hex[] = "0123456789abcdef";

    for (int i = NONCE_LENGTH - 1; i >= 0; --i) {
        pdf[NONCE_OFFSET + i] = (unsigned char)hex[nonce & 0xf];
        nonce >>= 4;
    }
}

static void set_student_number(unsigned char *pdf, const char *id)
{
    memcpy(pdf + STUDENT_ID_OFFSET, id, STUDENT_ID_LENGTH);
}
```

3 Tasks

1. **Nonce insertion and file I/O.** Implement functions to load a PDF into memory, overwrite its nonce and student number fields in place (see `set_nonce` and `set_student_number` above), and write out a “solved” copy of the file once a collision has been found. Your program should take a pair of input files (and, optionally, a thread count) as arguments, and should not require any other part of the file to be modified.
2. **Serial birthday attack.** Implement a single-threaded program that, given a pair of input PDFs, searches for a pair of nonces (n_1, n_2) such that `toy_hash(file1 with nonce  $n_1$ )` equals `toy_hash(file2 with nonce  $n_2$ )`.
3. **Parallel birthday attack.** Implement a parallel version of the above using OpenMP. You should parallelise the generation and hashing of nonce trials, and think carefully about how multiple threads can safely insert into and query a shared collision-detection data structure (e.g. using criticals or atomics, thread-local tables that are periodically merged, or a partitioned hash table). Clearly document the design choice you make and why.
4. **Correctness.** Ensure your program correctly verifies that a discovered pair of nonces produces a genuine collision (i.e. recompute `toy_hash` on the final, modified files and confirm the two hashes match) before writing out your solved files. Test your implementation on all of the provided example pairs, which vary in difficulty.
5. **Timing your solution.** Time only the search/collision-detection portion of your program (excluding file I/O). Run your parallel implementation on Kaya, varying the number of threads, and record how long each provided pair takes to solve. Remember that your job must complete within 15 minutes per pair on a single Kaya node. You may wish to repeat smaller runs several times.
6. **Report.** Write a $1,000 \pm 100$ word report describing your approach. Your report should include (at least) the following points:
 - A description of your birthday-attack algorithm and the collision-detection data structure you used.
 - A description of how you parallelised the search with OpenMP, including how you avoided or managed race conditions between threads.
 - A discussion of the memory requirements of your approach, and any trade-offs you made between memory usage and speed.
 - Thorough performance metrics and analysis, including speedup and scalability across the provided pairs of varying difficulty, and across different thread counts.

4 Submission

You should submit your assignment via LMS. Your submission must be a single zip/tar file containing your source code, a [Makefile](#), your report as a PDF, and all pairs of files you successfully broke. Your uploaded files should include the nonces in their headers so that each pair produces the same hash. Please also include a **README** explaining how to run your code.

All files must include the names and student numbers of both group members. You must also fill in the student number field in the header of every PDF you submit (see Table 1).

Your single **zip/tar file** should include:

- All of your source code, including any Makefiles or Slurm scripts.
- Your “solved” pairs of files, containing the solution nonces and your student number in their headers.
- Your $1,000 \pm 100$ word report describing your approach and performance metrics.

If we cannot build your project by simply running `make` on Kaya, you may receive zero for correctness.

5 HPC resources

While you should do basic development and debugging locally, we expect you to use Kaya for analysing the execution time of your solution for your report.

You should attempt to find collisions for all files using up to 96 cores using a single node on Kaya. If a single collision is taking longer than 15 minutes with this many cores, you should reconsider your strategy rather than submitting very long jobs to find a solution. If your submission indicates that solving any of the files took longer than 15 minutes, you may be deducted marks.

6 Marking Rubric

Criteria	Proficient (100%)	Competent (66%)	Novice (33%)	Marks
Nonce insertion and file I/O	Correct implementation fit to specifications. Only bytes 16–31 and 45–52 are modified; solved files verify correctly. Code was not messy, well documented, and ran successfully.	Correct implementation fit to specifications, but code was hard to follow.	Code required minor modifications to run, or did not follow specifications correctly (e.g. modified bytes outside the nonce or student number field).	1
Serial birthday attack	Correct, efficient serial implementation exploiting the birthday paradox. Code was not messy, well documented, and ran successfully.	Correct implementation exploiting the birthday paradox, but code was hard to follow or inefficient.	Implementation contained major correctness issues.	2
Parallel birthday attack: correctness	Parallel implementation always finds a genuine, verified collision for every provided pair.	Parallel implementation finds correct collisions for most provided pairs.	Parallel implementation is unreliable, non-deterministic in a harmful way, or frequently produces incorrect results.	2
Parallel birthday attack: use of OpenMP	Correct implementation with excellent, well-justified use of OpenMP for parallelism.	Correct implementation with basic OpenMP parallelism.	Implementation contained minor mistakes or did not make effective use of OpenMP (e.g. significant serial bottlenecks, poor scaling).	3
Correct solution of sample files	All file pairs solved correctly and within 15 minutes.	One file pair unsolved.	Multiple unsolved file pairs.	3

Criteria	Proficient (100%)	Competent (66%)	Novice (33%)	Marks
Description of the birthday-attack algorithm	Thorough analysis with excellent detail, including the collision-detection data structure and expected number of trials.	Good analysis, but lacking minor details.	Brief analysis, significantly lacking details.	3
Description of parallelisation and synchronisation strategy	Thorough analysis with excellent detail, including how race conditions on the shared data structure were avoided or managed.	Good analysis, but lacking minor details.	Brief analysis, significantly lacking details.	5
Description of memory usage and trade-offs	Thorough analysis with excellent detail, including the memory footprint of the collision-detection structure and any speed/memory trade-offs made.	Good analysis, but lacking minor details.	Brief analysis, significantly lacking details.	3
Performance metrics and analysis	Thorough analysis with excellent detail, including timings and speedup across thread counts and across pairs of varying difficulty.	Good analysis, but lacking minor details.	Brief analysis, significantly lacking details.	5
Formatting and general presentation	Well presented, easy to follow.	Okay presentation, but minor issues.	Poorly presented, hard to read or follow.	3
Total				30